Update 8: libfabric Under the Stack, From Queue Pairs to vLLM

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

October 2026 (draft: built and tested; cluster and GPU tables pending)

Contents

1 Summary **2**

2 Why libfabric **2**

3 Mapping the pool onto libfabric **3**

4 Provider behavior found while building **3**

5 libfabric under vLLM **4**
5.1 FabricBlobChannel 4
5.2 ScuffedRdmaConnector 4
5.3 NIXL's libfabric backend as the baseline 4

6 The rest of the stack **5**

7 Corrections **5**

8 Experiments **5**
8.1 E1. Hot-path latency by stack 6
8.2 E2. Cold bandwidth 6
8.3 E3. Head-of-line blocking, single vs dual endpoint 6
8.4 E4. libfabric configuration matrix 6
8.5 E5. Completion signaling 6
8.6 E6. Blob channel 6
8.7 E7. vLLM disaggregated serving on one GPU 7
8.8 E8. vLLM disaggregated serving across nodes 7

9 What the comparison can and cannot show **7**

10 Next steps **7**

1 Summary

Update 7 placed libscuffedrdma among the libfabric, UCX and DOCA stacks and noted that libfabric's endpoint model matches the hot/cold split directly. This update moves the middleware onto libfabric and carries it up to vLLM.

What exists now:

- `libscuffedfabric`: the dual pool on libfabric. Each class (hot, cold) gets its own endpoint and CQs, and can use its own provider and endpoint type (`FI_EP_MSG` or `FI_EP_RDM`).
- `libscuffedverbs`: the same C API on raw libibverbs RC queue pairs, so both stacks run one benchmark source.
- `FabricBlobChannel`: a Python blob channel over the pool, and `ScuffedRdmaConnector`, a vLLM V1 KV connector that moves disaggregated-prefill KV through it.
- A baseline path through vLLM's `NixlConnector` with NIXL's own `LIBFABRIC` backend, next to the usual UCX one.
- Around the transport: unified telemetry, a WFA with learned weights, the rule-tree policy engine, the PMP storage arm and an NVMe tier (Section 6).
- Nine Jupyter notebooks. Each runs in a local mode (smoke tests on any machine) and a cluster or GPU mode that produces the numbers.

Everything below was built and run end to end on libfabric 1.17's `tcp`, `tcp;ofi_rxm` and `shm` providers, and the connector passes its tests against vLLM 0.24.0's own base class. None of those runs are results. The tables are filled by notebooks 01 to 03 and 06 on Cerberus and Chimera, and by notebook 08 on a single desktop GPU.

2 Why libfabric

1. **The endpoint model fits the design.** libfabric exposes connected (`MSG`) and reliable-unconnected (`RDM`) endpoints, completion counters as a lighter alternative to CQs, selective completion, and wait objects on CQs [1, 2, 3]. Hot traffic wants the cheapest completion; cold traffic wants a deep queue and a thread that sleeps. With libibverbs both are built by hand. With libfabric they are configuration.
2. **GPFS runs on it.** IBM Storage Scale uses libfabric under its RDMA path, so a cold class on the same stack is the natural route for KV spill to a GPFS tier (Update 6).
3. **The serving stack is already moving there.** NIXL ships a libfabric backend that vLLM can select [5, 6], NVIDIA's Dynamo setup for AWS selects it for EFA [7], and EFA itself is reached through libfabric [8]. fabric-lib, a point-to-point RDMA library for LLM systems, builds its API on one-sided `WriteImm` with an `ImmCounter` for completion and reaches 400 Gbps on ConnectX-7 and EFA [9]. Those are the same two primitives this pool uses (`fi_writedata` and the remote-write counter in E5).
4. **The stack choice is measurable.** Smith's MUG'23 comparison concludes that libfabric and UCX "both provide very similar performance benefits" [10]. EPCC's ARCHER2 study found little difference in OSU micro-benchmarks but large application-level differences in `MPI_Alltoallv` [11]. Micro-benchmarks alone do not settle it, so this update measures under mixed traffic and at the connector level too.

Concept	libibverbs (libscuffedverbs)	libfabric (libscuffedfabric)
Class channel	RC QP per class	endpoint per class, FI_EP_MSG or FI_EP_RDM
Shared resources	one PD, one MR	one fabric + domain + fi_mr_reg when providers and types match
Hot completion	busy-polled send CQ	busy-polled CQ, or fi_cntr with FI_SELECTIVE_COMPLETION
Cold completion	send CQ + completion channel	CQ with FI_WAIT_UNSPEC, fi_cq_sread
Notification	RDMA_WRITE_WITH_IMM (32-bit imm)	fi_writedata, cq_data_size bytes (4 on verbs, 8 on tcp/shm)
Receive slots	posted recv WRs	posted fi_recv, required under FI_RX_CQ_DATA
Remote addressing	virtual address + rkey	virtual address or offset (FI_MR_VIRT_ADDR), provider or requested key
Connection setup	QPN/PSN/GID over a TCP side channel	MSG: passive endpoint, connect param selects the class; RDM: fi_av_insert
Remote-write counting	none	FI_RMA_EVENT counter, endpoint or MR bound
Progress	NIC / rxe kernel thread	provider-defined: auto (verbs) or manual (ofi_rxm, shm)

Table 1: The same pool in both libraries. Both export `scuffed_fabric.h`.

3 Mapping the pool onto libfabric

The mixed configuration from Update 7, hot on an RDM endpoint and cold on a connected one, is a run-time option (`--hot-prov 'verbs;ofi_rxm' --hot-ep rdm --cold-prov verbs --cold-ep msg`). It needs two domains, so the arena is registered twice. On verbs, `ofi_rxm` runs its own eager and rendezvous protocol on top of RC [4], which puts a protocol layer back on the hot path. E4 measures whether that costs more than it saves.

`sv_bench` is `sf_bench.c` compiled against `libscuffedverbs`. Payload offsets, warmup, window, timing (`CLOCK_MONOTONIC_RAW`), percentile code and JSON schema are identical, and the verbs library checks `wc.status` on every completion.

4 Provider behavior found while building

From libfabric 1.17's `tcp`, `tcp;ofi_rxm` and `shm` providers in a two-core container. Whether the verbs provider shares any of these is part of the cluster run.

- **`tcp` needs target-side CQ reads despite `FI_PROGRESS_AUTO`.** With the target not reading the cold CQ, a 1 MB write stream stalled after about three writes. The benchmark server now runs a yielding progress thread on the cold endpoint for every provider.
- **Remote-write counters are not uniform.** `tcp` advertises `FI_RMA_EVENT` but did not increment an endpoint-bound counter, and `fi_mr_enable` on an `FI_RMA_EVENT` MR returned `-FI_ENOSYS`. `ofi_rxm` does not offer it. `shm` counts through the endpoint. The library binds both ways and the benchmark probes one write each way before trusting a counter.
- **Connection-request info drops optional caps.** The passive side's `FI_CONNREQ` info lacked `FI_RMA_EVENT` although the listener asked for it. The library copies it back.
- **`ofi_rxm` connects on first use.** The initiator's first write spins on `-FI_EAGAIN` until the target makes progress. Posting is bounded (60 s), so an idle peer becomes an error instead of a hang.
- **`shm` does not advertise write-after-write ordering** for RMA, so the sentinel mechanism is skipped there.
- **With one endpoint the target cannot tell classes apart.** The single-endpoint baseline therefore measures exactly the coupling the dual pool removes.

5 libfabric under vLLM

5.1 FabricBlobChannel

The connector needs "send this named blob, receive named blobs", not raw writes. `FabricBlobChannel` provides that over the pool with credit-based slots. Each blob goes with `fi_writedata` into a receiver slot on the class the sender picks (cold by default, hot when the WFA tags it), with the slot number and a DATA tag in the immediate. The receiver returns a credit as an 8 B write on the hot endpoint (CREDIT tag + slot). Credits and control blobs do not share an endpoint with bulk KV, which is the head-of-line property the thesis is about, applied to the connector's own protocol. A `TcpBlobChannel` with the same interface is the fallback and the baseline.

5.2 ScuffedRdmaConnector

An out-of-tree `KVConnectorBase_V1` loaded through `kv_connector_module_path`. The proxy sends the prompt to prefill with `max_tokens=1` and a transfer id. Each attention layer's `save_kv_layer` gathers that request's paged blocks, copies them to host memory, encodes them (raw bfloat16, or scuffedQuant for the cold path) and queues one blob per layer. The WFA tags each blob hot or cold. Decode reports the prompt as externally matched, waits per layer, and scatters into its own paged cache. Layers that never arrive go back to vLLM through `get_block_ids_with_load_errors` and are recomputed. If the libfabric channel cannot open, the connector falls back to TCP and records a `tcp_fallback` telemetry event.

Staging through host memory is a deliberate choice for this hardware: Tower 2's Proxmox kernel lacks `CONFIG_INFINIBAND_PEER_MEM`, SoftRoCE has no GPUDirect path, and neither does a desktop GPU under WSL2. It costs a device-to-host copy per layer. E7 and E8 show how much of TTFT that is.

The connector was written against vLLM `620e8924` (April 2026). Its tests now also pass against vLLM 0.24.0, the version a fresh install resolves today: method signatures against the 0.24.0 source, import and subclass check against the installed package, and a producer to consumer run through vLLM's own `KVConnectorBase_V1` (CPU, no model).

5.3 NIXL's libfabric backend as the baseline

vLLM reads NIXL backends from a `backends` list in `kv_connector_extra_config`, default UCX [6]. Setting it to `LIBFABRIC` is a one-line change. That makes the stock stack's libfabric path a direct baseline: same model, same proxy, same sweep, different connector. The launcher has it as `CONNECTOR=nixl-libfabric`.

Reading the plugin at NIXL commit `d642316` (22 Oct 2026) [5]:

- Provider discovery asks for `FI_EP_RDM` with `FI_MSG | FI_RMA` and takes `cxi`, then `efa`, then `tcp`, then `sockets`. `verbs` is not in that list.
- Its README lists libfabric 1.21 or newer and GPUDirect RDMA as requirements, and AWS EFA as the only validated hardware.

So on Cerberus and Chimera (SoftRoCE) and on the towers (ConnectX-4 without peer memory), NIXL's libfabric backend can at most run over libfabric `tcp`, and may not start at all. If it does not start, the error is the result and goes in the table as such. The contrast is the point: the libfabric path the serving stack ships is shaped around EFA and Slingshot with GPUDirect, while this pool targets verbs on commodity RoCE with host staging.

6 The rest of the stack

Built this month, around the transport. All of it has tests and a notebook.

- **Telemetry** (`middleware/telemetry.py`). One `SignalVector` per transfer (size, role, phase, QP class, queue depth, completion latency, inter-arrival), per-class counters including QP errors, reconnects and TCP fallbacks, log-bucketed latency histograms, and Prometheus export. The pyverbs pool, the libfabric pool and the connector all report into it, so the classifier, the policy engine and the evaluation read the same numbers.
- **Learned WFA** (`wfa_learner.py`, notebook 04). Transition weights fit by gradient descent on `SignalVector` features, with phase-by-size terms so each phase can have its own size cutoff. On synthetic traces the static threshold classifier routes most decode KV deltas cold: they fall in its 4 KB to 256 KB band, where it routes by access count, and most have not been seen often enough to go hot. The learned weights recover the labeling rule. That is a synthetic result and partly by construction, since the oracle defines decode deltas as hot. The real test is training on telemetry dumps from the cluster runs, labeled by measured completion latency.
- **Rule-Node Expression Trees** (`middleware/policy/`). Boolean trees over signals and context (GPU memory, link utilization), tried in priority order, deciding route (hot, cold, spill) and compression. Classifiers plug in as leaves, so the WFA is one input to the policy, not the policy.
- **PMP storage arm** (`pmp_controller.py`, notebook 07). A third bang-bang arm for spill that activates above a GPU-memory watermark, with hysteresis. Below the watermark the two-arm controller from Update 4 is unchanged.
- **NVMe tier** (`storage_tier.py`, notebook 07). Spill and retrieve with coalesced `pwrite`, CRC-checked reads, extent reuse, optional `O_DIRECT`, gated by the policy or PMP decision.
- **scuffedQuant wire format.** Indices are now bit-packed and `CompressedKV.nbytes` reports packed bytes. At head_dim 64, 3 bits and a 256-bit QJL sketch, that is 2× against FP16.
- **NCCL net plugin** (`middleware/nccl_plugin/`) builds. An NCCL and vLLM run on it is still open.

7 Corrections

- The 8 μs cross-node figure is withdrawn. The Python pool from Update 4 did not check completion status, so failed or flushed writes counted as fast completions, including 8 MB writes that "completed" in about 8 μs, which 10GbE cannot do. The pool now raises on a bad status and notebook 01 re-runs it.
- The scuffedQuant ratio quoted as 4× against FP16 was against FP32. It is 2× against FP16 (above).
- Several README rows (SoftRoCE bandwidth, single-QP p50, dual-QP overhead, FA3 speedup) cited sources that do not hold them. They are removed and re-measured where they matter.

8 Experiments

E1 to E6 and E8: Cerberus as client or prefill, Chimera as server or decode, SoftRoCE on 10GbE, hot and cold threads pinned away from the NIC interrupts. Repeat E1 to E5 on Tower 1 and Tower 2 over ConnectX-4. Every number traces to a JSON file that records the exact commands, `fi_info --version` and the provider string per class.

8.1 E1. Hot-path latency by stack

`sig -M cq`, one-way latency (RTT/2), 8 B to 64 KB. Notebook 03.

	64 B		4 KB	
Stack	p50 (μs)	p99 (μs)	p50 (μs)	p99 (μs)
libibverbs RC (native)	*pending*	*pending*	*pending*	*pending*
libfabric verbs MSG	*pending*	*pending*	*pending*	*pending*
libfabric verbs;ofi_rxm RDM	*pending*	*pending*	*pending*	*pending*

Table 2: Hot-path latency, SoftRoCE, Cerberus $\leftrightarrow$ Chimera.

8.2 E2. Cold bandwidth

`bw`, window 16, 4 KB to 16 MB, with and without a wait object on the cold CQ. Notebook 03.

8.3 E3. Head-of-line blocking, single vs dual endpoint

Hot 64 B and 4 KB ping-pong while cold streams 256 KB, 1 MB and 4 MB writes, window 8. The claim holds on a stack if dual-endpoint hot p99 under load stays near its unloaded p99 while single-endpoint p99 grows with cold size. If both grow, the bottleneck is below the queue pair (for SoftRoCE, the rxe kernel thread), which is itself a result. Notebook 01.

Stack, endpoints	hot p99, no cold	hot p99, 1 MB cold	cold MB/s
libibverbs, single QP	*pending*	*pending*	*pending*
libibverbs, dual QP	*pending*	*pending*	*pending*
libfabric verbs MSG, single EP	*pending*	*pending*	*pending*
libfabric verbs MSG, dual EP	*pending*	*pending*	*pending*
libfabric verbs;ofi_rxm RDM, dual EP	*pending*	*pending*	*pending*

Table 3: Hot 64 B round trip (μs) under a cold stream.

8.4 E4. libfabric configuration matrix

Under the same mixed load: MSG/MSG, RDM/RDM, hot RDM + cold MSG, hot completions through `fi_cntr`, cold CQ with a wait object, and the single-endpoint baseline. Notebook 03.

8.5 E5. Completion signaling

CQ data, flag in payload, sentinel write, remote-write counter, and write followed by an 8 B read (the GPFS flush pattern), each with user-space instructions retired per post, following Papadopoulou et al.'s per-operation accounting [12]. The counter row is the libfabric form of fabric-lib's `ImmCounter`. Adopt a mechanism only if it moves p99. Notebook 02.

8.6 E6. Blob channel

`FabricBlobChannel` against `TcpBlobChannel`, 4 KB to 32 MB, cross node, one acknowledgment per blob. This is the transport under the connector, so it links E1 to E5 to E7 and E8.

8.7 E7. vLLM disaggregated serving on one GPU

Notebook 08, on a desktop GPU (WSL2) or one lab node alone. `scripts/local_gpu/setup_gpu_env.sh` installs libfabric, CUDA torch and vLLM, builds the native libraries and runs the tests. Prefill and decode instances share the card (Granite 3.3-2B, 0.40 of memory each) and talk through `FabricBlobChannel` on libfabric `shm`. Checks:

- the connector's data path with KV caches on the GPU, byte-exact over TCP, libfabric `shm` and libfabric `tcp`;
- greedy outputs from the disaggregated pair against a single instance, token for token;
- TTFT and ITL for both, plus the same sweep through `NixlConnector` with the UCX and `LIBFABRIC` backends when NIXL is installed;
- scuffedQuant on real Granite KV (per-layer top-8 overlap, mean and spread) and GPU decompression cost.

These are single-host numbers. They show the connector works inside real vLLM with real GPU tensors and what disaggregation costs on one card. They are not transport results.

Setup (one GPU)	TTFT p50, 2K prompt (ms)	ITL p50 (ms)	greedy match
single instance	*pending*	*pending*	reference
ScuffedRdmaConnector, libfabric shm	*pending*	*pending*	*pending*
NixlConnector, UCX	*pending*	*pending*	*pending*
NixlConnector, LIBFABRIC	*pending*	*pending*	*pending*

Table 4: Single-GPU end-to-end run (notebook 08).

8.8 E8. vLLM disaggregated serving across nodes

Notebook 06. Prefill on Cerberus, decode on Chimera, the same TTFT/ITL sweep (128 to 4096 prompt tokens) against five setups: single node, `ScuffedRdmaConnector` on libfabric verbs, the same connector on TCP, `NixlConnector` on UCX, and `NixlConnector` on its `LIBFABRIC` backend over `tcp`. This is the experiment the thesis question is asked at.

9 What the comparison can and cannot show

- On SoftRoCE the verbs provider and native verbs reach the same rxe device, so their difference is libfabric's software path. The ConnectX-4 runs separate software cost from a hardware path that is no longer the bottleneck.
- rxe processes packets in a kernel thread. If it saturates, separate QPs share it and dual-endpoint gains shrink regardless of library. E3 shows whether that happens.
- NIXL's libfabric backend runs here over `tcp` at best, so E8 compares this pool against NIXL's libfabric path as it behaves on commodity RoCE, not on EFA.
- E7 runs on one host through shared memory. It isolates connector and vLLM overhead; it says nothing about the network.

10 Next steps

1. Run notebook 08 on the desktop GPU and fill E7.
2. Run notebooks 00 to 03 on Cerberus and Chimera, then the towers. Fill E1 to E5.

3. Run notebook 06 across nodes for E8, including the NIXL `LIBFABRIC` row.
4. If hot RDM + cold MSG wins E4, make it the default and route NVMe and GPFS spill through the same libfabric domain.
5. Train the learned WFA on the telemetry dumps from these runs, labeled by measured latency, and compare against the static classifier on real traces.

References

[1] P. Grun, S. Hefty, S. Sur, D. Goodell, R. D. Russell, H. Pritchard, and J. M. Squyres, "A Brief Introduction to the OpenFabrics Interfaces: A New Network API for Maximizing High Performance Application Efficiency," *IEEE Hot Interconnects*, 2015.

[2] OpenFabrics Interfaces Working Group, `fi_endpoint(3)`, `fi_cq(3)`, `fi_mr(3)` man pages, libfabric 1.17. https://ofiwg.github.io/libfabric/

[3] OpenFabrics Interfaces Working Group, `fi_cntr(3)` man page, libfabric. https://ofiwg.github.io/libfabric/

[4] OpenFabrics Interfaces Working Group, `fi_rxm(7)` and `fi_verbs(7)` man pages, libfabric. https://ofiwg.github.io/libfabric/

[5] NVIDIA, NIXL libfabric plugin, `src/plugins/libfabric/README.md` and `src/utils/libfabric/libfabric_common.cpp`, commit `d642316`, 2026. https://github.com/ai-dynamo/nixl

[6] vLLM, `NixlConnector` worker, `vllm/distributed/kv_transfer/kv_connector/v1/nixl/base_worker.py`, v0.24.0. https://github.com/vllm-project/vllm

[7] NVIDIA Dynamo documentation, "EFA on AWS." https://docs.nvidia.com/dynamo/latest/kubernetes/installation/rdma-setup/efa-on-aws

[8] L. Chen, "Harnessing 3200 Gbps Network (1): RDMA and EFA," 2024. https://le.qun.ch/en/blog/2024/12/25/libfabric-efa-1-rdma/

[9] N. Licker, K. Hu, V. Zaytsev, and L. Chen, "fabric-lib: RDMA Point-to-Point Communication for LLM Systems" (formerly TransferEngine), arXiv:2510.27656v2, 2026.

[10] B. Smith, "Omni-Path and the Open Fabrics Interfaces," Cornelis Networks, MVAPICH User Group (MUG) 2023.

[11] M. Bareford, D. Henty, W. Lucas, and A. Turner, "OpenFabrics and UCX Performance on the ARCHER2 HPE Cray EX System," Cray User Group (CUG) 2022. Data: https://github.com/ARCHER2-HPC/performance_ofi-ucx

[12] N. Papadopoulou, L. Oden, and P. Balaji, "A Performance Study of UCX over InfiniBand," *Proc. IEEE/ACM CCGrid*, 2017.